

RENT-EQUIP PRO: Arriendo de Maquinaria Pesada

Proyecto 6 (Renting / Servicios) • Arquitectura Django REST Framework + PostgreSQL + 3FN

Estudiante:	Sebastián Torres Zamorano	Docente:	Marcelo Patricio Alvarado Aravena
Sección:	AP-N4-C1	Ponderación:	25% Total (70% Defensa Oral)
Asignatura:	Desarrollo Backend (EVA-2)	Proyecto:	N°6 Arriendo Maquinaria (3FN)
Motor DB:	PostgreSQL 12 (Nativo)	Seguridad:	JWT Claims RBAC + ACID Trans.

1. Dominio de la Arquitectura, Configuración DB y Modelos (12 Puntos)

P1.1: ¿Por qué este modelo de datos cumple con la Tercera Forma Normal (3FN)? Justifique técnicamente.

Respuesta Senior: Una base de datos está en 3FN si está en 2FN y **ningún atributo no clave depende transitivamente de la clave primaria** ($X \rightarrow Y$ no transitivo). En nuestro sistema:

- 1FN:** Cada campo es atómico (sin listas serializadas dentro de celdas), y existen claves primarias definidas.
- 2FN:** Todos los atributos no clave dependen funcionalmente de la clave primaria completa de cada tabla.
- 3FN:** Desacoplamos las tarifas del catálogo mutable respecto al contrato histórico. La tabla `DetalleContratoArriendo` congela los valores (`tarifa_diaria_congelada`, `monto_garantia_congelado`, `dias_totales`). Esto garantiza que si el administrador modifica mañana el precio de catálogo de una Excavadora CAT, los contratos previamente firmados y pagados no sufran alteraciones no autorizadas ni dependencias transitivas corruptas.

P1.2: ¿Cómo se configura PostgreSQL en Django y qué ventajas aporta en este proyecto?

Respuesta Senior: En `config/settings.py` se define el motor nativo `django.db.backends.postgresql` conectado a la base de datos `arriendo_maquinaria_db` mediante las librerías `psycopg2-binary` o `psycopg` (v3).

Ventaja Crítica: A diferencia de SQLite (que bloquea todo el archivo en escrituras concurrentes), PostgreSQL soporta **bloqueo a nivel de fila (Row-Level Locking)** mediante `SELECT ... FOR UPDATE`. Esto permite que múltiples empresas constructoras hagan reservas simultáneas sin provocar condiciones de carrera ni sobre-arriendo de unidades físicas de la flota.

P1.3: ¿Dónde y por qué se implementan los atributos explícitos con CHOICES?

Respuesta Senior: Se implementaron enumeraciones tipadas usando `models.TextChoices` en tres entidades clave:

- `RolUsuario.choices`: `EMPRESA_CONSTRUCTORA` vs `EJECUTIVO_ARRIENDOS`.
- `EstadoContrato.choices`: `PENDIENTE`, `PAGADO`, `ENTREGADO`, `COMPLETADO`, `CANCELADO`.
- `EstadoMaquinaria.choices`: `DISPONIBLE`, `MANTENCION`, `DE_BAJA`.

Esto asegura integridad de dominio a nivel de ORM y de base de datos, evitando strings arbitrarios y facilitando la documentación OpenAPI automática.

2. Flujo de Autenticación JWT, Claims de Rol y Permisos RBAC (12 Puntos)

P2.1: Explique el ciclo de vida del JWT y cómo se inyectan los claims personalizados.

Respuesta Senior: El usuario envía sus credenciales a POST /api/auth/login/. La clase CustomTokenObtainPairSerializer hereda de TokenObtainPairSerializer de SimpleJWT y sobrescribe el método de clase get_token(cls, user):

```
token['user_id'] = user.id
```

```
token['rol'] = user.rol # 'EMPRESA_CONSTRUCTORA' o 'EJECUTIVO_ARRIENDOS'
```

```
token['razon_social'] = user.razon_social
```

El cliente recibe un **Access Token** (vida útil de 8 horas) y un **Refresh Token** (7 días con rotación y token blacklist). Al enviar solicitudes protegidas, el cliente incluye el header Authorization: Bearer <access_token>.

P2.2: ¿Cómo se aplican las restricciones de permisos RBAC en los endpoints?

Respuesta Senior: Creamos clases de permisos personalizadas que heredan de rest_framework.permissions.BasePermission:

- IsEmpresaConstructora: Valida request.user.rol == 'EMPRESA_CONSTRUCTORA'. Protege el carro de arriendos, el checkout y mis-contratos.
- IsEjecutivoArriendos: Valida request.user.rol == 'EJECUTIVO_ARRIENDOS' o is_staff. Protege la mutación de catálogo y la transición de estados de contratos.
- IsAdminOrReadOnly: Permite safe methods (GET) a todo público y restringe escrituras a ejecutivos.
- IsDocAdminUser: Cumple con el requerimiento de clase restringiendo Swagger al rol de Administrador.

3. Persistencia del Carro de Compras Post-Logout (12 Puntos)

P3.1: ¿Cómo se garantiza que el carro persista tras cerrar sesión o cambiar de PC?

Respuesta Senior: El carro **no reside en cookies ni en localStorage**. Está modelado en PostgreSQL mediante la entidad CarroArriendo que mantiene una relación OneToOneField(Usuario, related_name='carro_activo') y una relación 1 a N con ItemCarroArriendo. Cada ítem almacena la maquinaria FK, cantidad, fecha de inicio y fecha de fin.

Al hacer logout o conectarse desde otro dispositivo móvil o navegador, la autenticación JWT consulta el registro persistente del usuario en la base de datos, recuperando íntegramente los ítems y las cotizaciones en tiempo real.

P3.2: ¿Por qué el inventario NO se descuenta al agregar un ítem al carro?

Respuesta Senior: Descontar stock al agregar al carro provocaría *ataques de denegación de inventario (Cart Abandonment Stock Hijacking)*, donde un usuario malicioso llena su carro sin pagar bloqueando la flota para clientes reales. Por regla de negocio y rúbrica EVA-2, el carro solo almacena la intención de arriendo; el descuento ocurre de forma atómica y estricta en el momento del **Checkout y Pago Confirmado**.

4. Ciclo Transaccional Atómico, Checkout y Gestión de Flota (12 Puntos)

P4.1: Explique la ejecución de la transacción atómica en el Checkout ('POST /api/contratos/checkout/').

Respuesta Senior: Se encapsula la lógica dentro del context manager with transaction.atomic(): de Django:

1. **Bloqueo Pesimista:** Se ejecuta `Maquinaria.objects.select_for_update().filter(id__in=maquinarias_ids)` para bloquear las filas en PostgreSQL.
 2. **Validación de Disponibilidad:** Se comprueba `flota_disponible >= item.cantidad` y `estado_operativo == 'DISPONIBLE'`.
 3. **Descuento Atómico:** Se resta `maq.flota_disponible -= item.cantidad` y se ejecuta `maq.save(update_fields=['flota_disponible'])`.
 4. **Creación de Contrato:** Se genera el ContratoArriendo en estado PAGADO y los DetalleContratoArriendo congelando precios y fechas.
 5. **Vaciado de Carro y Auditoría:** Se limpia el carro activo y se registra en HistorialTransicionContrato.
- Si ocurre cualquier falla o stock insuficiente, se lanza una excepción y la base de datos realiza un **ROLLBACK automático**, dejando los saldos intactos.

P4.2: ¿Cómo opera la reposición automática de flota al transicionar a COMPLETADO o CANCELADO?

Respuesta Senior: En la vista `AdminActualizarEstadoContratoView` (exclusiva de Ejecutivos), al recibir un PATCH con estado COMPLETADO (devolución del equipo por la constructora) o CANCELADO, el sistema itera sobre los detalles del contrato y reincorpora las unidades con bloqueo atómico:

`maq.flota_disponible = min(maq.flota_total, maq.flota_disponible + detalle.cantidad)`

Esto restituye la maquinaria al catálogo público en tiempo real para que otras empresas puedan arrendarla de inmediato.

5. Filtros, Búsquedas y Documentación OpenAPI / Swagger (10 Puntos)

P5.1: ¿Cómo está implementado `django-filter` y qué parámetros acepta?

Respuesta Senior: Se construyó la clase `MaquinariaFilter(django_filters.FilterSet)` vinculada a `MaquinariaViewSet` a través de `filterset_class` y `DjangoFilterBackend`. Acepta:

- `categoria` (FK) y `categoria_slug` (`exact/lexact`).
- `min_precio` y `max_precio` (rango sobre `tarifa_diaria` con `gte / lte`).
- `solo_disponibles=true` (filtro booleano que comprueba `flota_disponible > 0`).
- `q` (búsqueda de texto sobre nombre, SKU, marca, modelo y descripción).

P5.2: ¿Cómo se generó la documentación Swagger y por qué está restringida?

Respuesta Senior: Se utilizó `drf-spectacular` configurado con esquema OpenAPI 3.0. Expone los endpoints `/api/schema/`, `/api/docs/` (Swagger UI) y `/api/redoc/`. Cada endpoint cuenta con decoradores `@extend_schema` con descripción, tags y códigos de respuesta.

Siguiendo la instrucción del docente en clase (*'header solo admin'*), la vista de documentación incluye un banner/control de acceso verificado para el rol Ejecutivo/Administrador.

6. Calidad de Código, Comentarios en Bloques y Footer Base (12 Puntos)

P6.1: ¿Cómo se asegura la renderización global del Footer con datos del estudiante?

Respuesta Senior: Se creó un **Context Processor personalizado** en `web_ui/context_processors.py` llamado `evaluacion_footer_info`, registrado en `settings.py` dentro de `TEMPLATES['OPTIONS']['context_processors']`.

Esto inyecta automáticamente el diccionario `alumno_info` (Nombre: **Sebastián Torres Zamorano**, Docente: **Marcelo Patricio Alvarado Aravena**, Sección: **AP-N4-C1**, Año: **2026**) en todas las vistas HTML del sistema sin ensuciar la lógica de los controladores.

7. Visión Estratégica: Marketing y Escalabilidad B2B SaaS (€100.000+)

Para posicionar esta plataforma en el **1% superior de la industria tecnológica de Renting** y justificar una valoración sobre los 100.000 Euros, se diseñó la siguiente arquitectura de escalamiento comercial:

1. Modelo de Monetización B2B Dual:

- *Take-Rate por Transacción (5% - 8%)*: Comisión sobre cada arriendo generado por empresas asociadas.
- *Suscripción SaaS Enterprise (€299 - €999/mes)*: Para constructoras con módulos avanzados de gestión de faena multi-obra.

2. **Telemetría e Integración IoT**: Conexión API vía MQTT / Webhooks con computadores a bordo de maquinaria CAT y Cummins para sincronizar horómetros en tiempo real y alertas de mantención predictiva preventiva.

3. **Fintech & Garantías Digitales (Smart Escrow)**: Sustitución de boletas de garantía bancarias físicas por pólizas de caución digitales integradas con pasarelas de pago B2B (Transbank Webpay Plus, Stripe B2B).

4. **Arquitectura Cloud Nativa**: Soporte para réplicas de lectura en PostgreSQL, caché Redis y despliegue en Kubernetes (EKS / GKE) con alta disponibilidad del 99.99%.